LAPORAN JOBSHEET
“10-Static Site Generation (SSG)”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

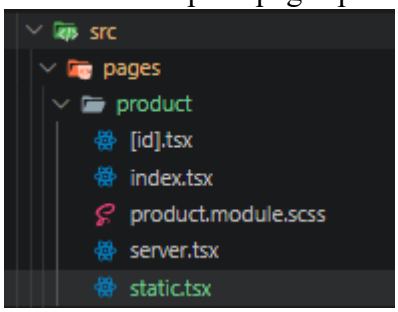
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

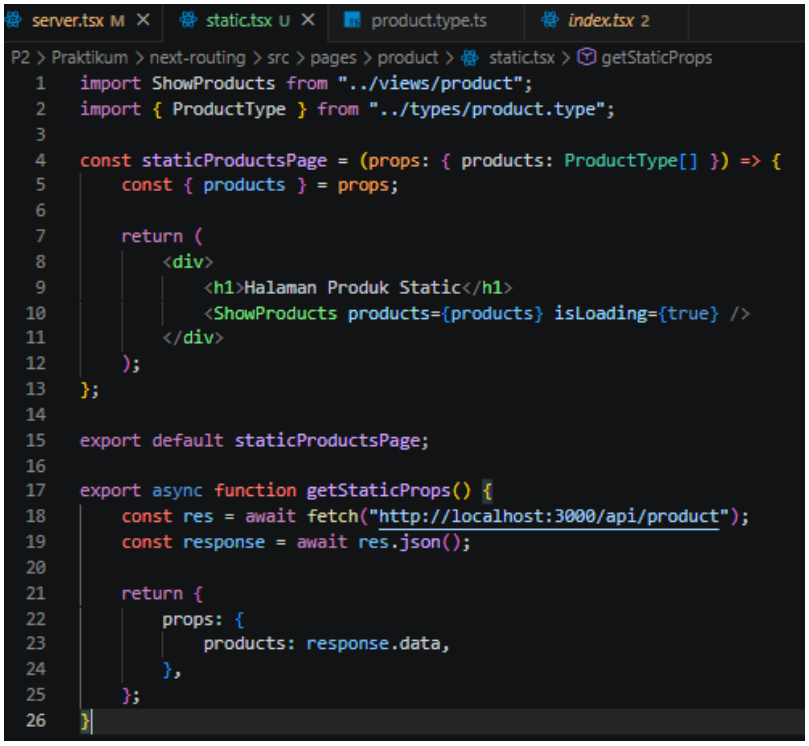
2026

Bagian 1 – Setup Halaman Static

1. Buat file baru pada pages/products/static.tsx

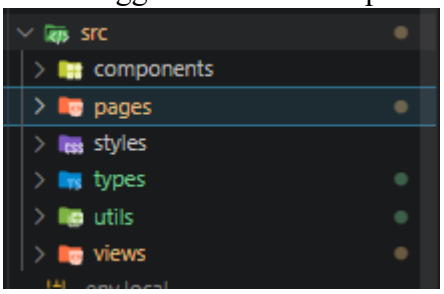


2. Modifikasi file static.tsx



Bagian 2 – Build Production Mode

1. Pindah beberapa folder diluar pages antara lain
   - Untuk menghindari error maka folder Views, utils, styles dipindah di luar folder src sehingga susunan folder pada src sebagai berikut



2. Jalankan: npm run build
   - Jalankan npm run dev dan pastikan ini jalan (jangan distop saat ngebuild), jadi buka dua terminal

Npm run build

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm run build

Route (pages)
┌ ○ /
├   /_app
├ ○ /404
├ ○ /about
├ ƒ /api/hello
├ ƒ /api/product
├ ○ /auth/login
├ ○ /auth/register
├ ○ /blog
├ ○ /blog/[slug]
├ ○ /category/[[...slug]]
├ ○ /product
├ ○ /product/[id]
├ ƒ /product/server
├ ● /product/static (652 ms)
├ ○ /profile
├ ○ /profile/edit
├ ○ /setting/app
├ ○ /shop/[[...slug]]
├ ○ /user
└ ○ /user/password

○  (Static)   prerendered as static content
●  (SSG)      prerendered as static HTML (uses getStaticProps)
ƒ  (Dynamic)  server-rendered on demand
```

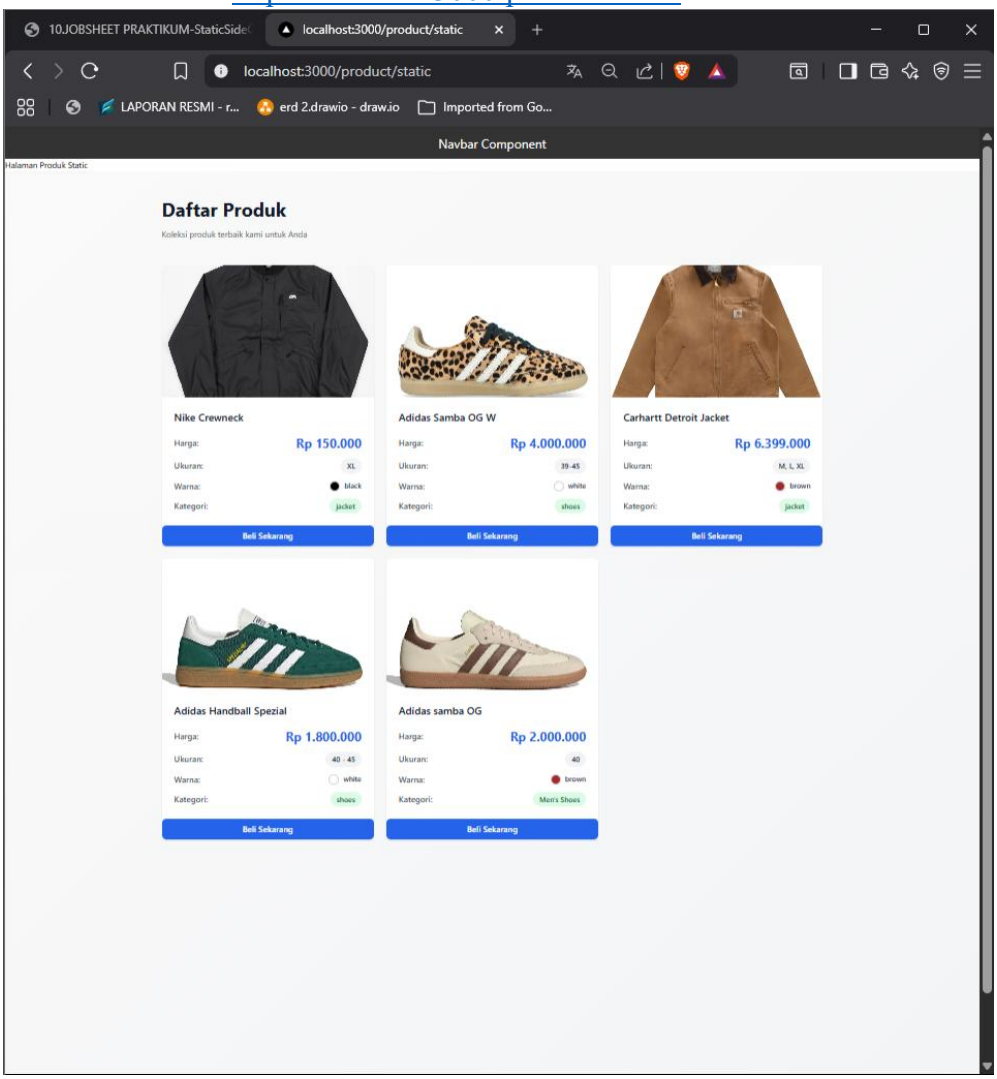
Npm run dev

```
○ PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm run dev

  > next-routing@0.1.0 dev
  > next dev

  ✓ Console Ninja extension is connected to Next.js, see https://tinyurl.com/2vt8jxzw
  ⚠ Console Ninja Turbopack support is a PRO feature, currently available to community users while in preview
  ▲ Next.js 16.1.6 (Turbopack)
  - Local:         http://localhost:3000
  - Network:       http://100.102.141.86:3000
  - Environments: .env.local

  ✓ Starting...
  ✓ Ready in 1244ms
  Fetching from collection: products
  Total documents found: 5
  Document 5hfeTIt7HIyrCmnokNa: {
    size: 'XL',
    image: 'https://cdn.shopify.com/s/files/1/1202/6102/products/nike-sb-seasonal-skate-jacket-black-black-black-white-1.jpg',
    name: 'Nike Crewneck ',
    color: 'black',
    category: 'jacket',
    price: 150000
```

3. Jika berhasil: npm run start

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm run start

> next-routing@0.1.0 start
> next start

▲ Next.js 16.1.6
- Local:         http://localhost:3000
- Network:       http://100.102.141.86:3000

✓ Starting...
✓ Ready in 1023ms
```

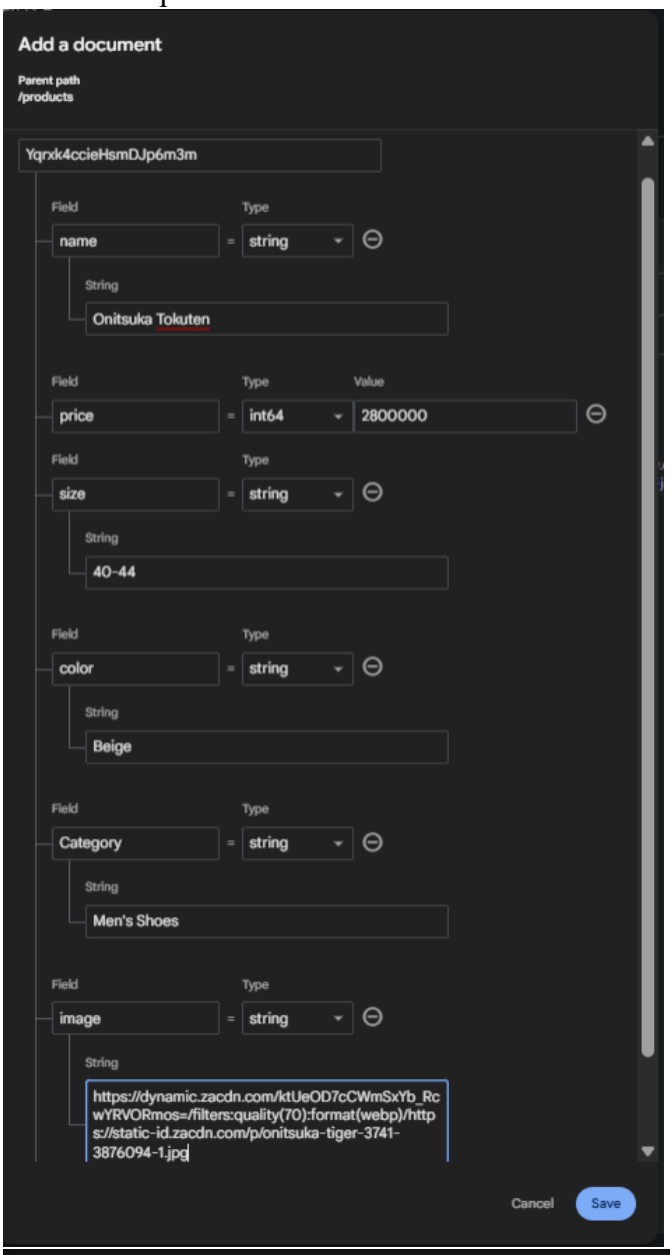
4. Jalankan browse: http://localhost:3000/product/static

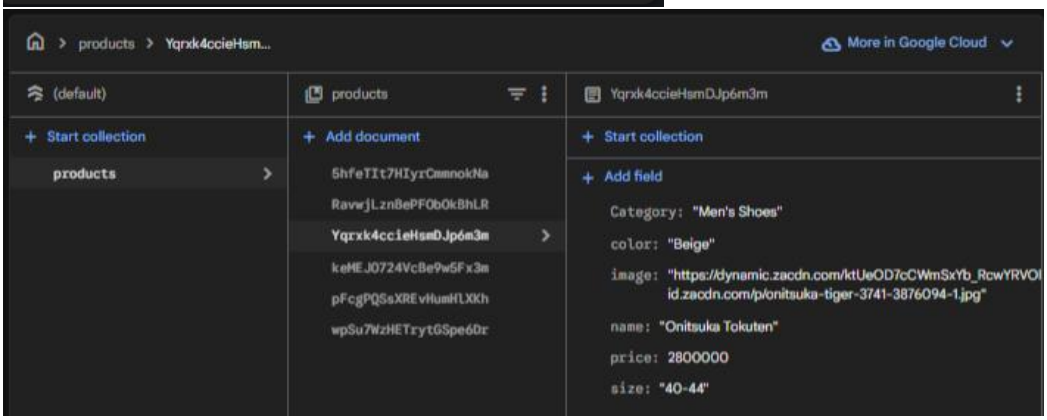


Bagian 3 – Pengujian Perubahan Data

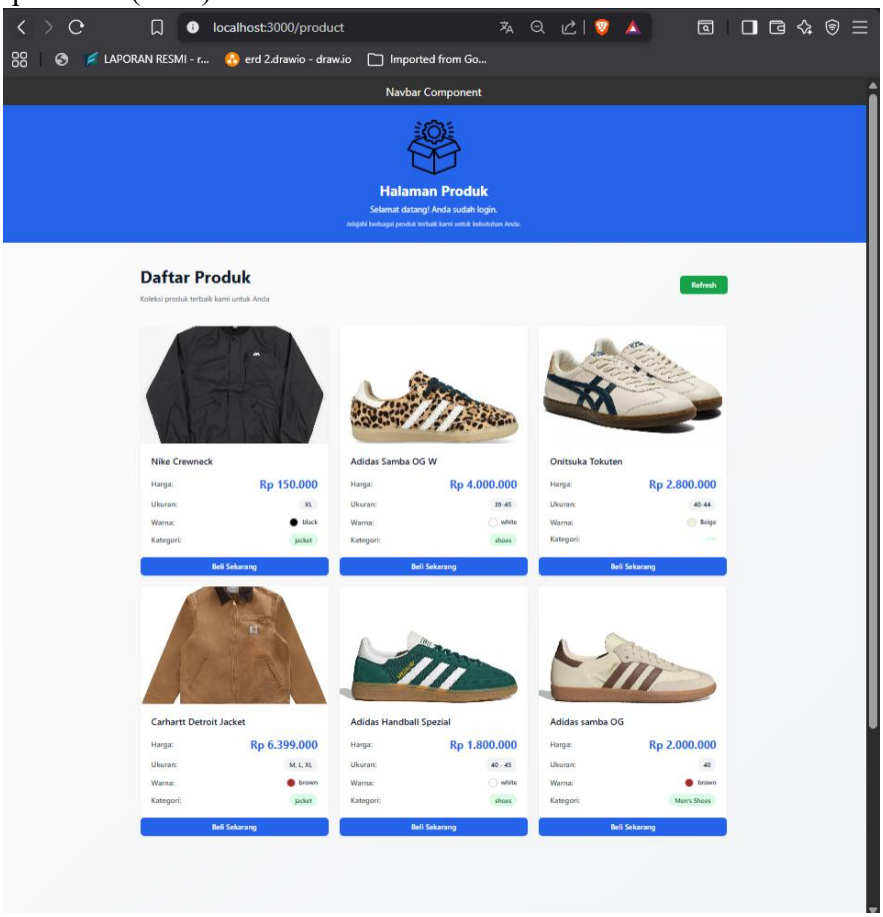
Uji 1 – Tambah Data di Database

1. Buka database firebasenya
   - Tambahkan produk baru di database.

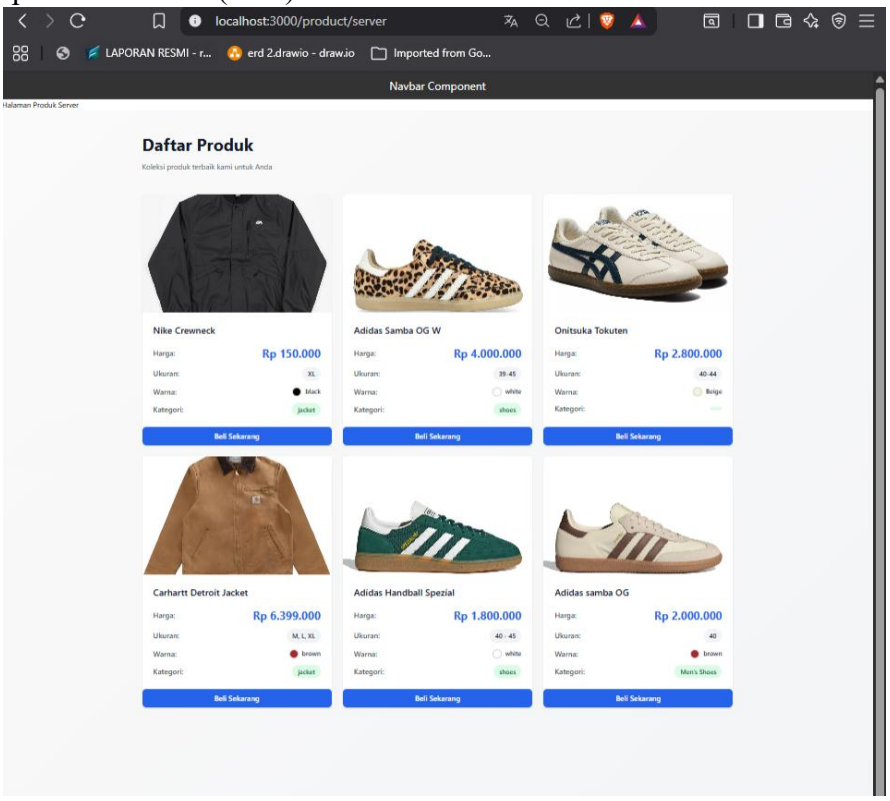




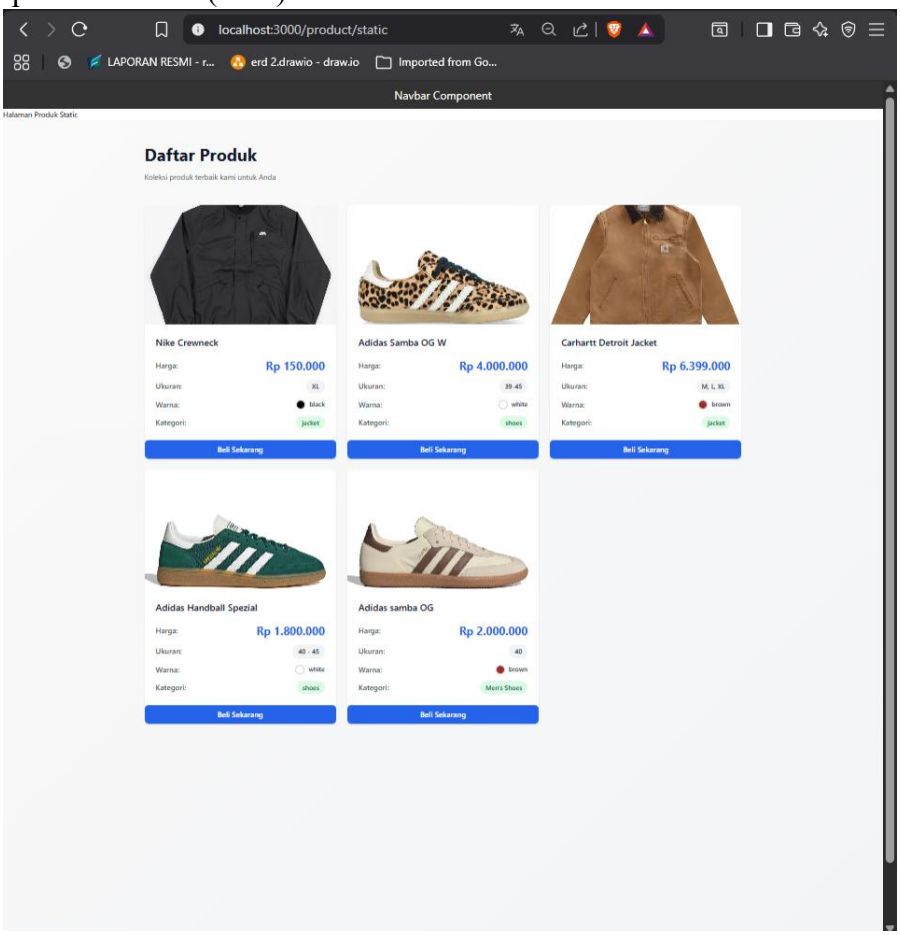
2. Buka halaman:
   - /products (CSR) → Data bertambah



   - /products/server (SSR) → Data bertambah



- /products/static (SSG) → Data tidak berubah



Uji 2 – Build Ulang

1. Jalankan kembali:
   - npm run build
     - lakukan secara bersamaan dengan npm run dev saat melakukan npm run build

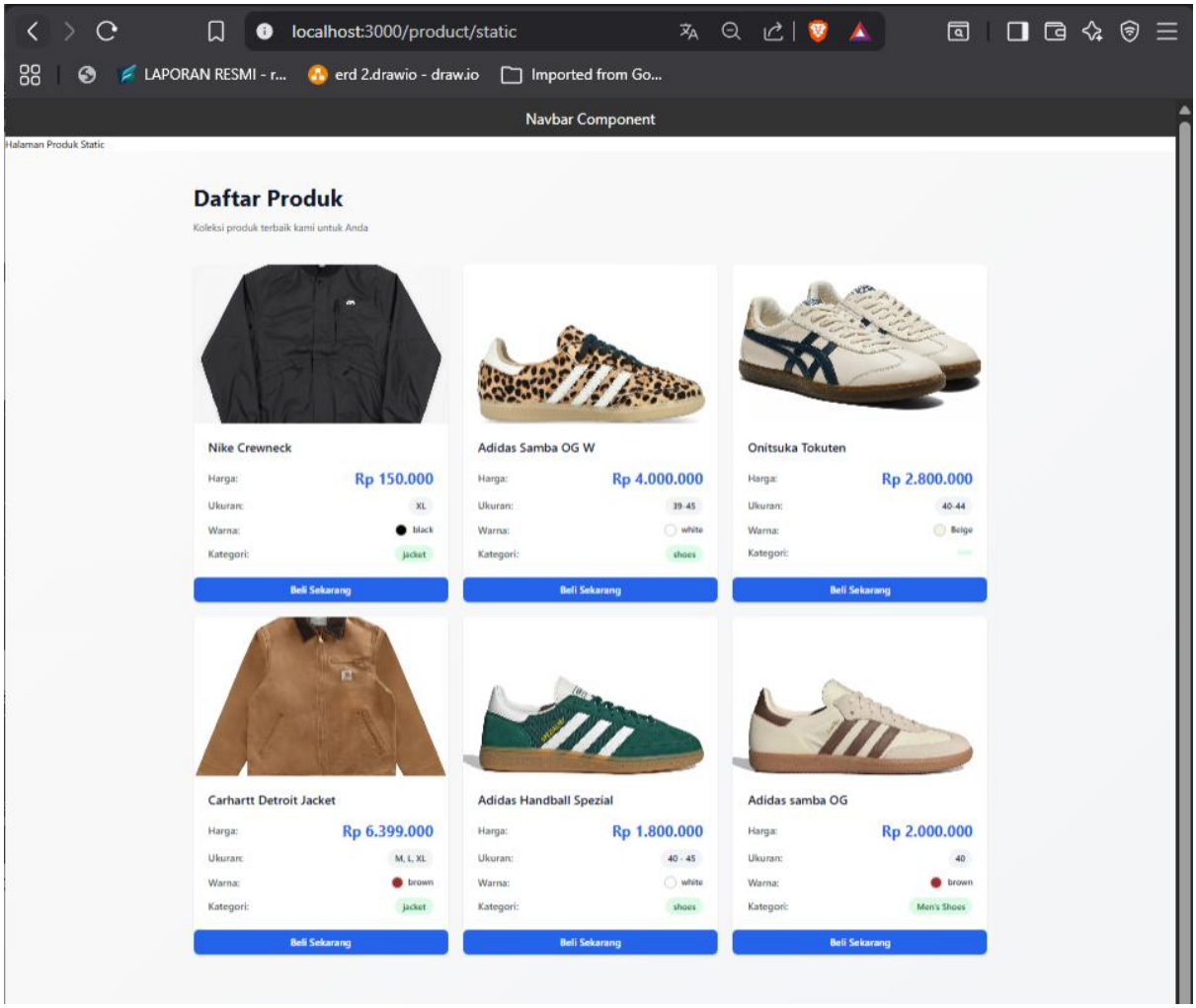
```
✓ Finalizing page optimization in 16.8ms

Route (pages)
┌ ○ /
│   /_app
├ ○ /404
├ ○ /about
├ ƒ /api/hello
├ ƒ /api/product
├ ○ /auth/login
├ ○ /auth/register
├ ○ /blog
├ ○ /blog/[slug]
├ ○ /category/[[...slug]]
├ ○ /product
├ ○ /product/[id]
├ ƒ /product/server
├ ● /product/static
├ ○ /profile
├ ○ /profile/edit
├ ○ /setting/app
├ ○ /shop/[[...slug]]
├ ○ /user
└ ○ /user/password

○  (Static)   prerendered as static content
●  (SSG)      prerendered as static HTML (uses getStaticProps)
ƒ  (Dynamic)  server-rendered on demand
```

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing>
                                                                                    npm run dev

  > next-routing@0.1.0 dev
  > next dev

  ✓ Console Ninja extension is connected to Next.js, see https://tinyurl.com/2vt8jxzw
  ⚠ Console Ninja Turbopack support is a PRO feature, currently available to community users while in preview
  ⚠ Port 3000 is in use by process 20744, using available port 3001 instead.
  ▲ Next.js 16.1.6 (Turbopack)
  - Local:         http://localhost:3001
  - Network:       http://100.102.141.86:3001
  - Environments: .env.local

  ✓ Starting...
  ✓ Ready in 1422ms
```

- npm run start
  - npm run dev stop terlebih dahulu setelah itu npm run start

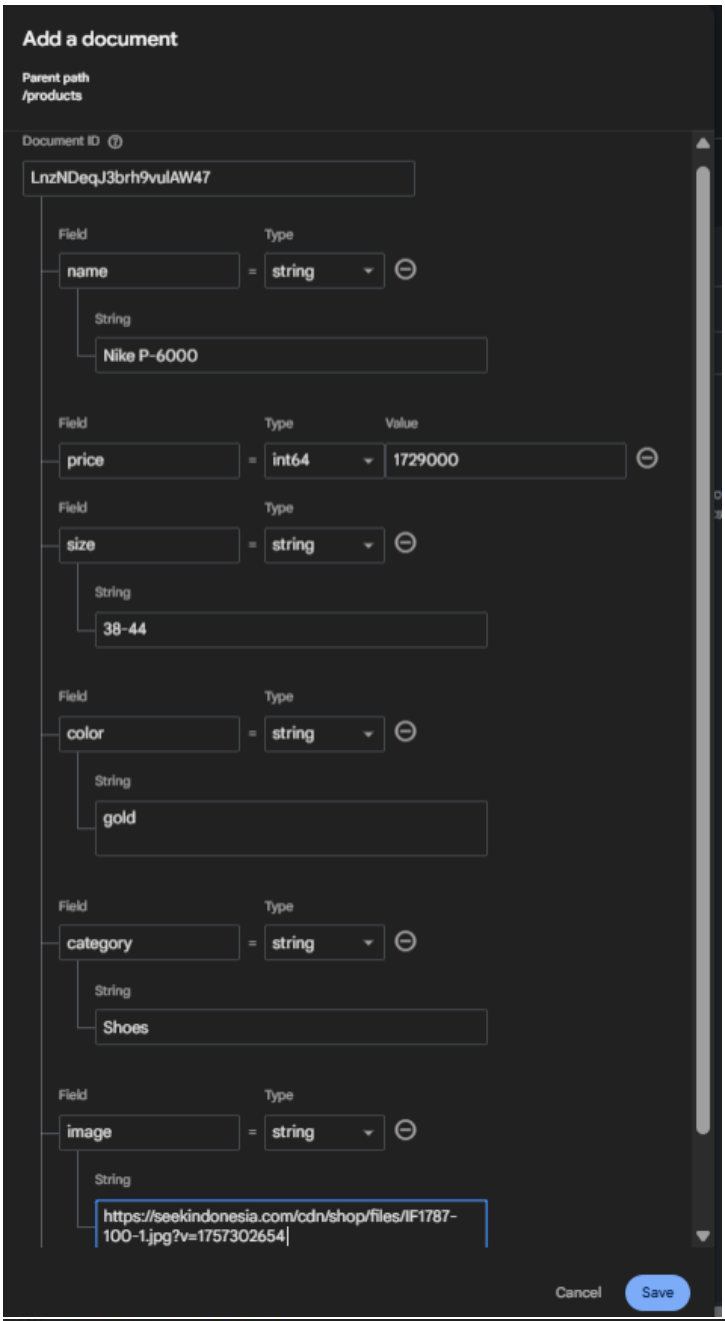
```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm run start

> next-routing@0.1.0 start
> next start

▲ Next.js 16.1.6
- Local:         http://localhost:3000
- Network:       http://100.102.141.86:3000

✓ Starting...
```

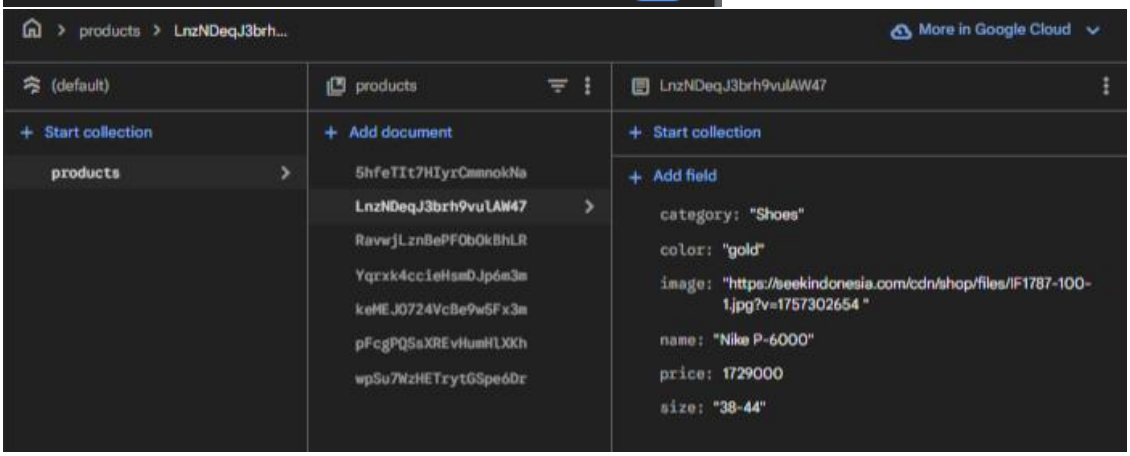
2. Refresh halaman static → Data baru muncul



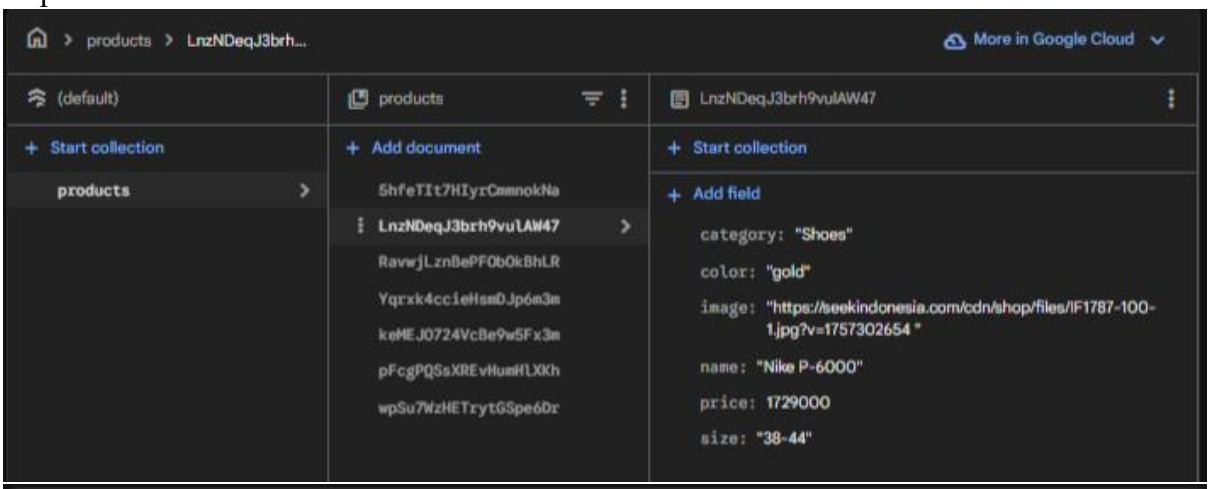
TUGAS PRAKTIKUM

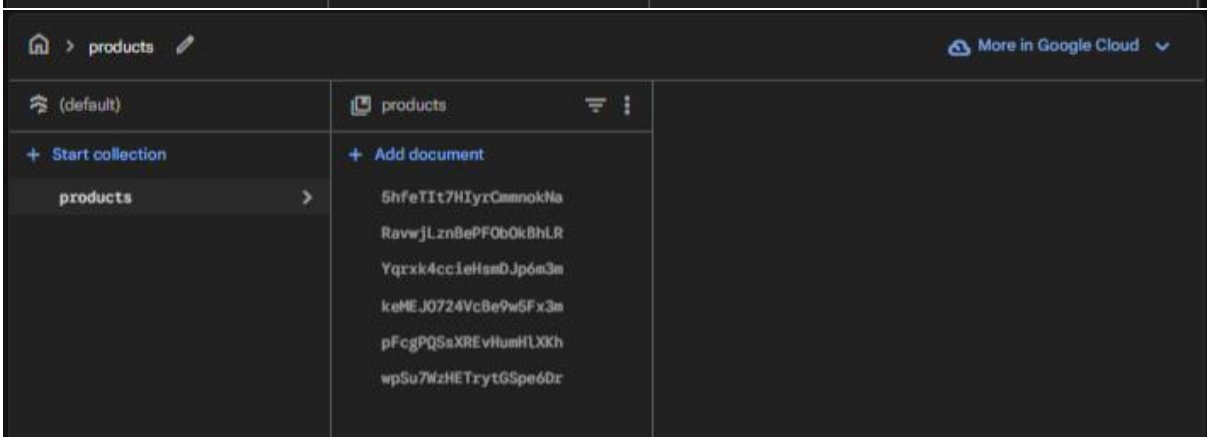
- Tambah data



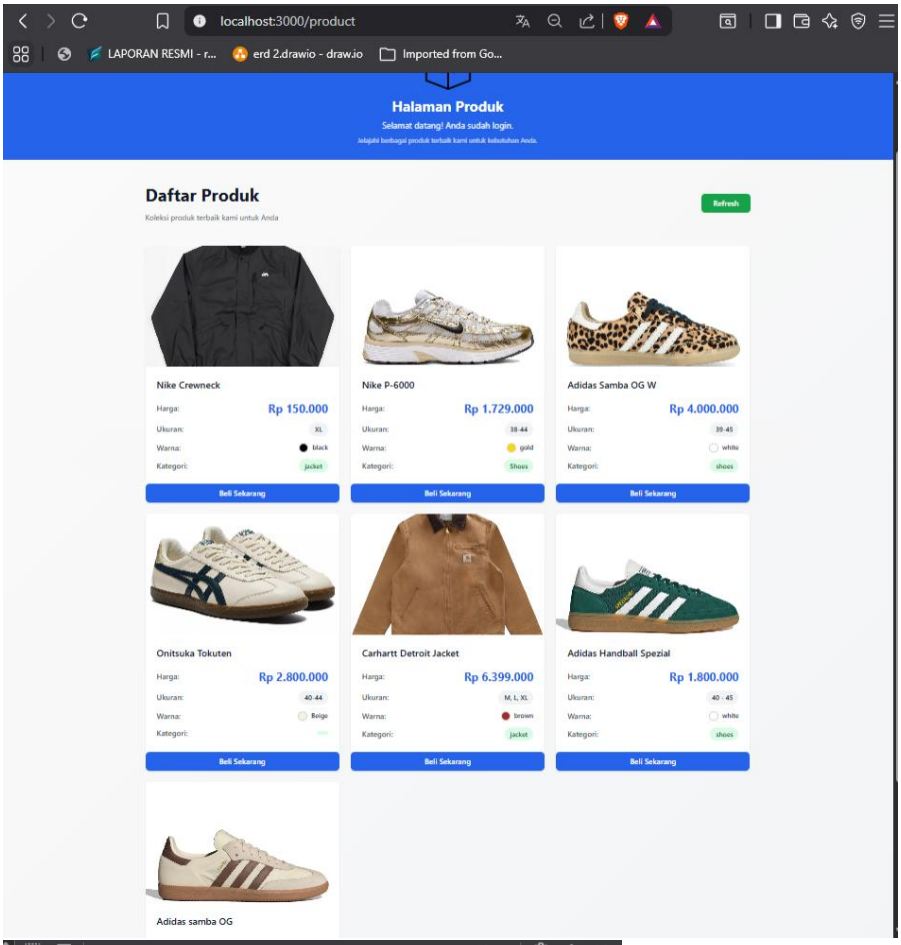


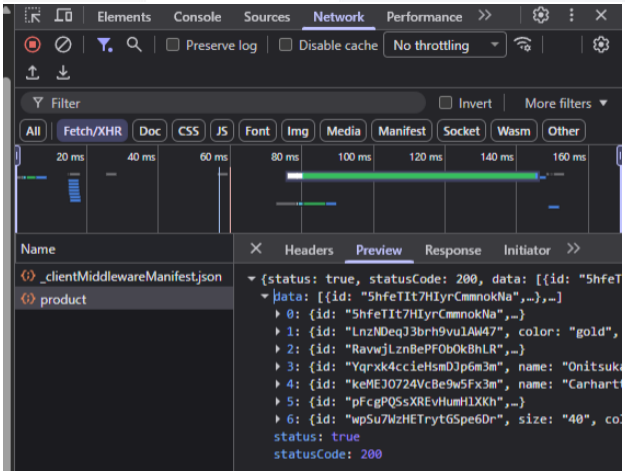
- Hapus Data


products > LnzNDeqJ3brh...
More in Google Cloud
(default)
products
LnzNDeqJ3brh9vulAW47
+ Start collection
+ Add document
+ Start collection
products
5hfeTIt7HIyrCmmnokNa
LnzNDeqJ3brh9vulAW47
RavwjLznBePFObOkBhLR
Yqrxk4ccieHsmDJp6m3m
keMEJO724VcBe9w5Fx3m
pFcgPQSsXREvHumHlXKh
wpSu7WzHETrytGSpe6Dr
+ Add field
category: "Shoes"
color: "gold"
image: "https://seekindonesia.com/cdn/shop/files/IF1787-100-1.jpg?v=1757302654 "
name: "Nike P-6000"
price: 1729000
size: "38-44"


products
More in Google Cloud
(default)
products
+ Start collection
+ Add document
products
5hfeTIt7HIyrCmmnokNa
RavwjLznBePFObOkBhLR
Yqrxk4ccieHsmDJp6m3m
keMEJO724VcBe9w5Fx3m
pFcgPQSsXREvHumHlXKh
wpSu7WzHETrytGSpe6Dr

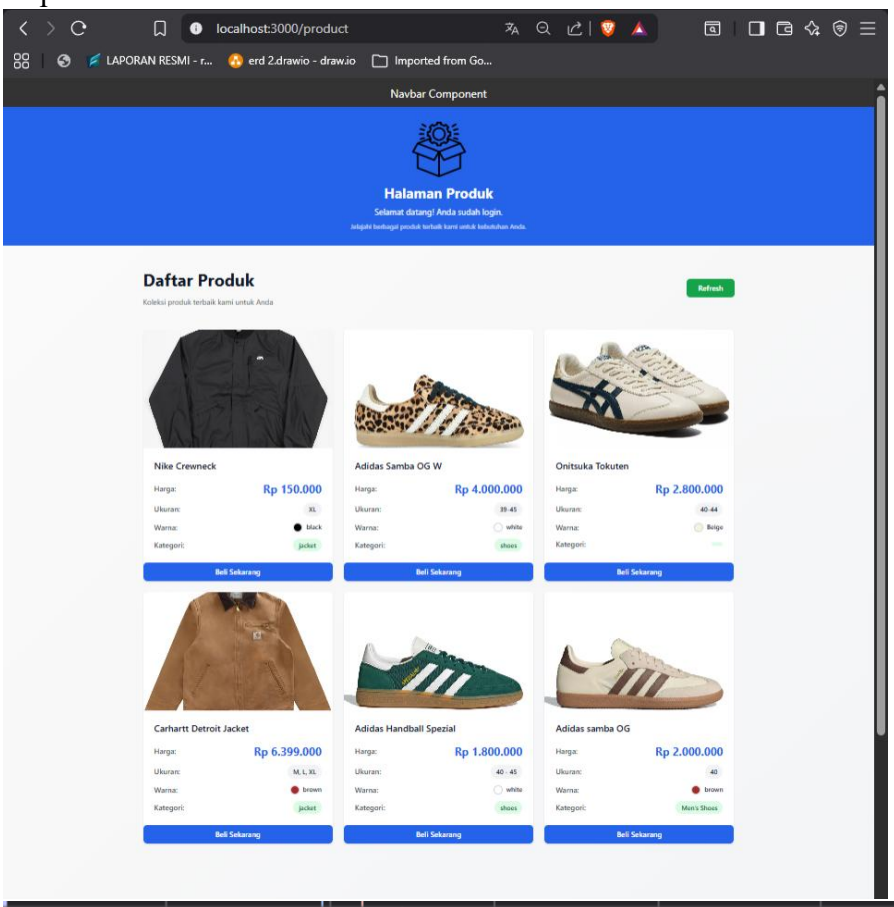
1. **CSR (*Client-Side Rendering*)**
   - Tambah data

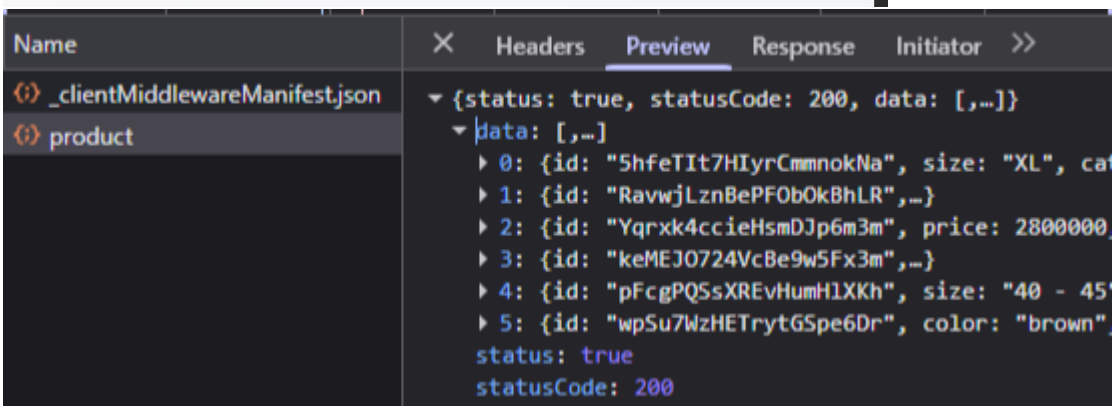




Otomatis ditambahkan ketika di refresh dan halaman CSR request (permintaan) menuju API (seperti /api/product) terlihat dengan sangat jelas.

- Hapus Data



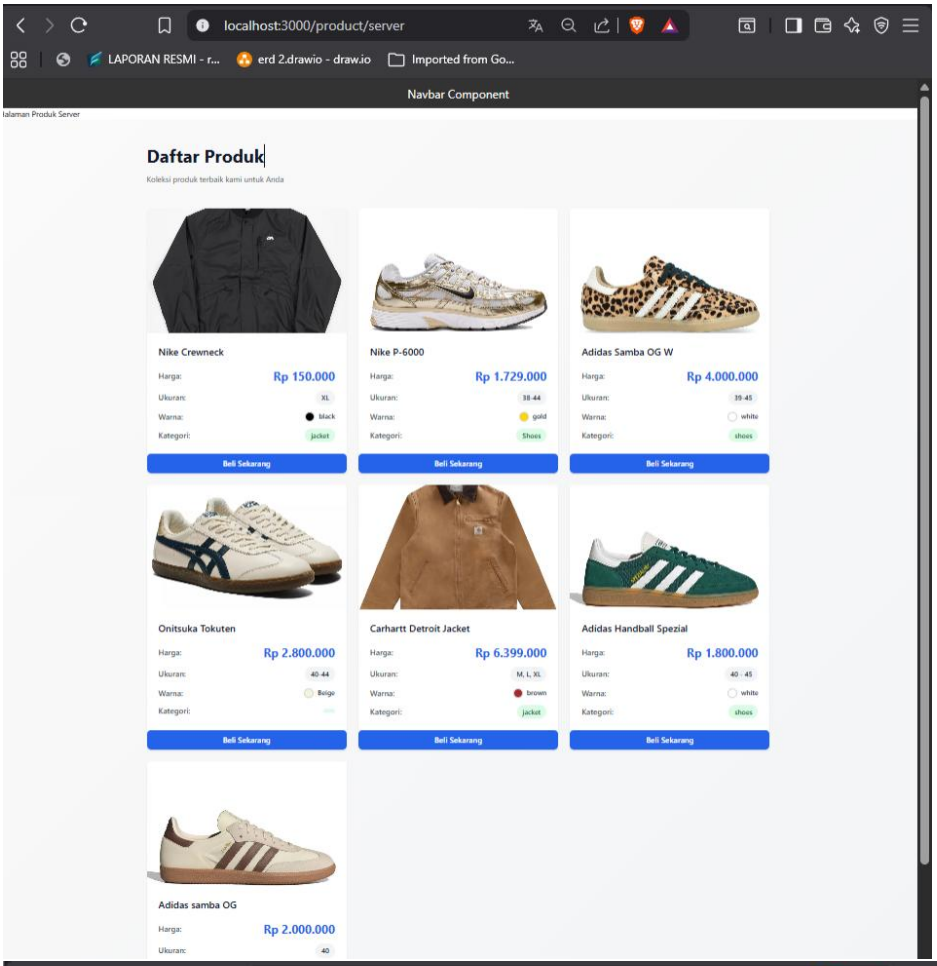


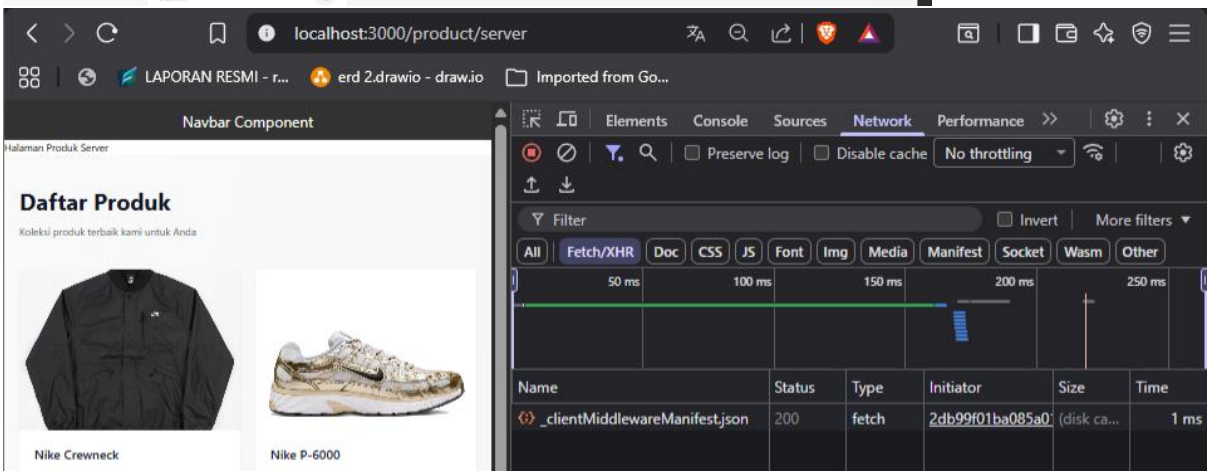
(Data ke-6 sudah tidak ada, tandanya sudah dihapus),

- Hasil

  Pada CSR (Client Side Rendering) di Next.js, data diambil langsung dari browser (client). Artinya saat halaman dibuka, JavaScript di browser akan memanggil API menggunakan fetch. Karena request itu dilakukan di sisi user, maka bisa dilihat di DevTools → Network (termasuk URL API, response, dll). Jadi kelihatan karena memang request-nya terjadi di depan (client-side).

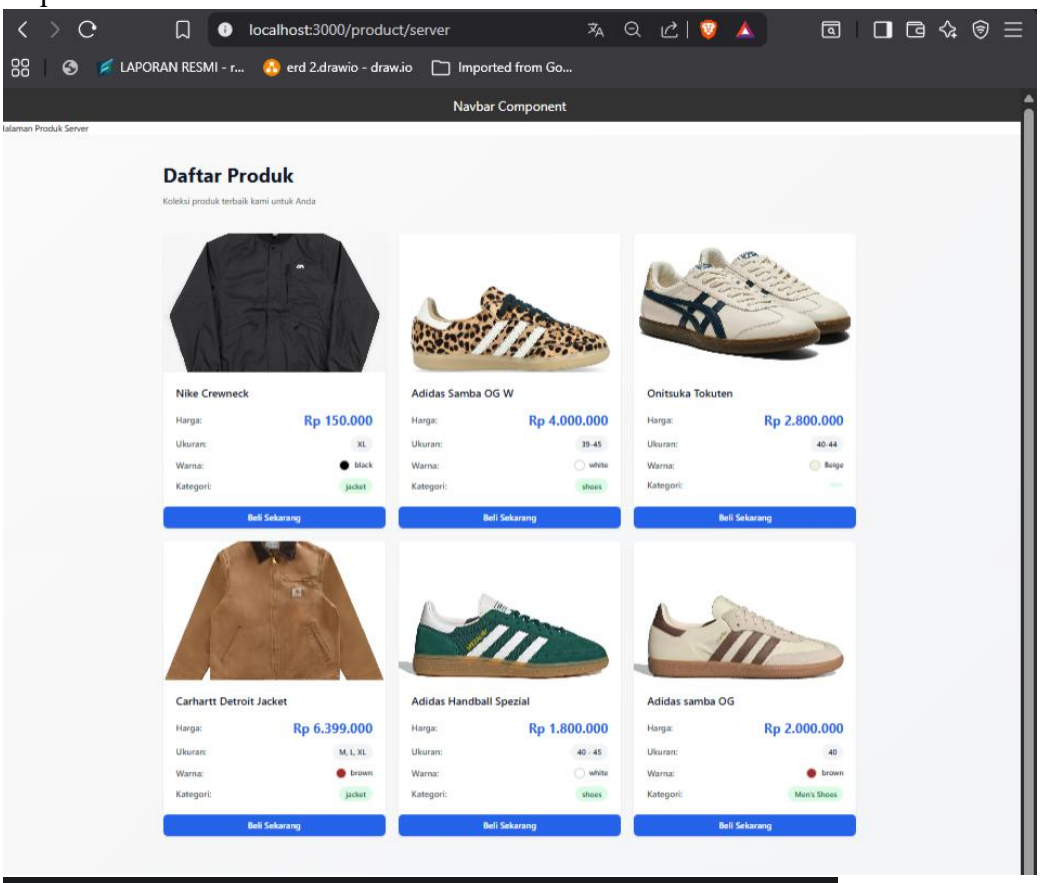
2. **SSR (*Server-Side Rendering*)**
   - Tambah data





Ketika halaman SSR di-refresh, request API tersebut tidak terlihat sama sekali dan tidak ada produk dengan type fetch yang terekam oleh peramban.

- Hapus Data



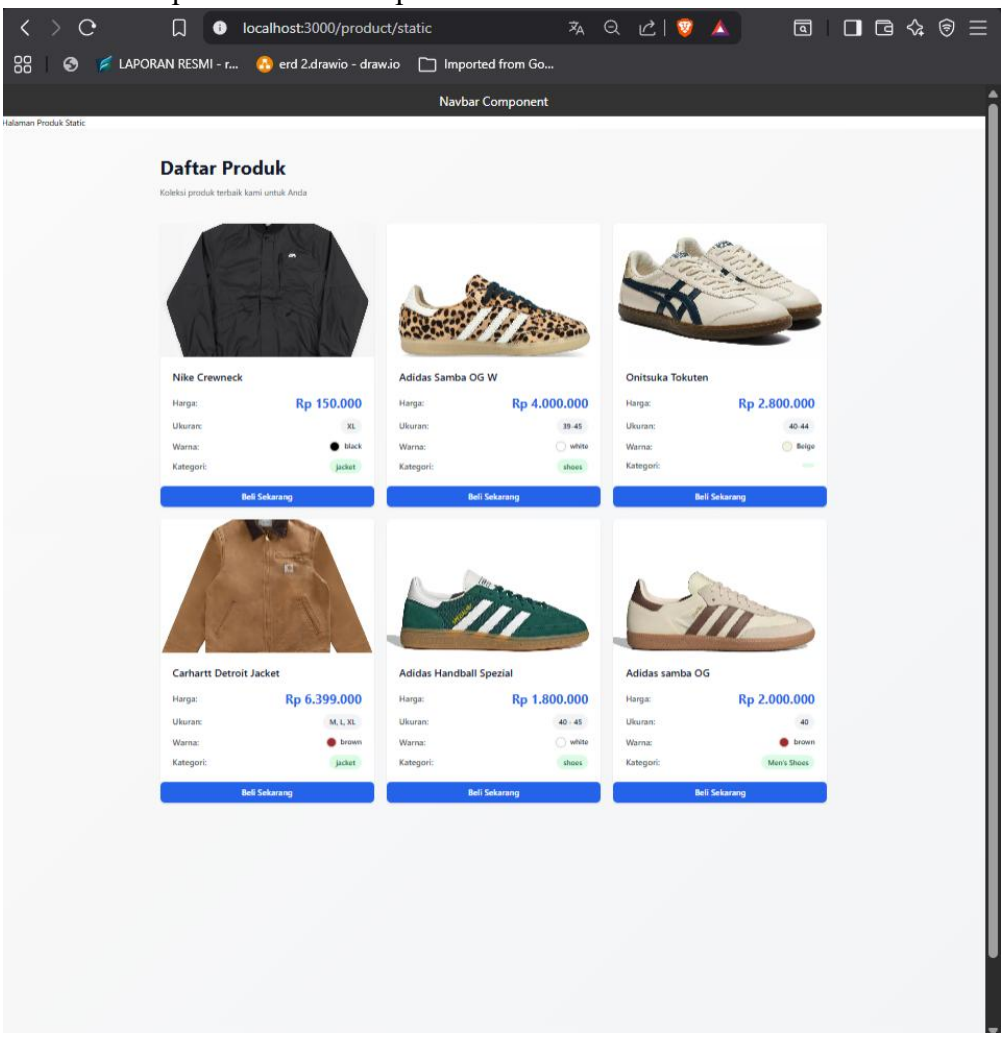
Dapat dilihat bahwa sudah tidak ada lagi Nike P-6000 jadi sudah terupdate, dapat dilihat dari terminal API nya karena client tidak bisa melihat API jika menggunakan SSR

- Hasil

SSR (Server Side Rendering), proses fetch data terjadi di server (misalnya lewat getServerSideProps). Jadi saat user membuka halaman, server sudah lebih dulu ambil data dari API, lalu mengirim HTML yang sudah jadi ke browser. Akibatnya, di browser kamu tidak melihat request ke API tersebut, karena request-nya terjadi "di belakang layar" (server-side), bukan dari client.

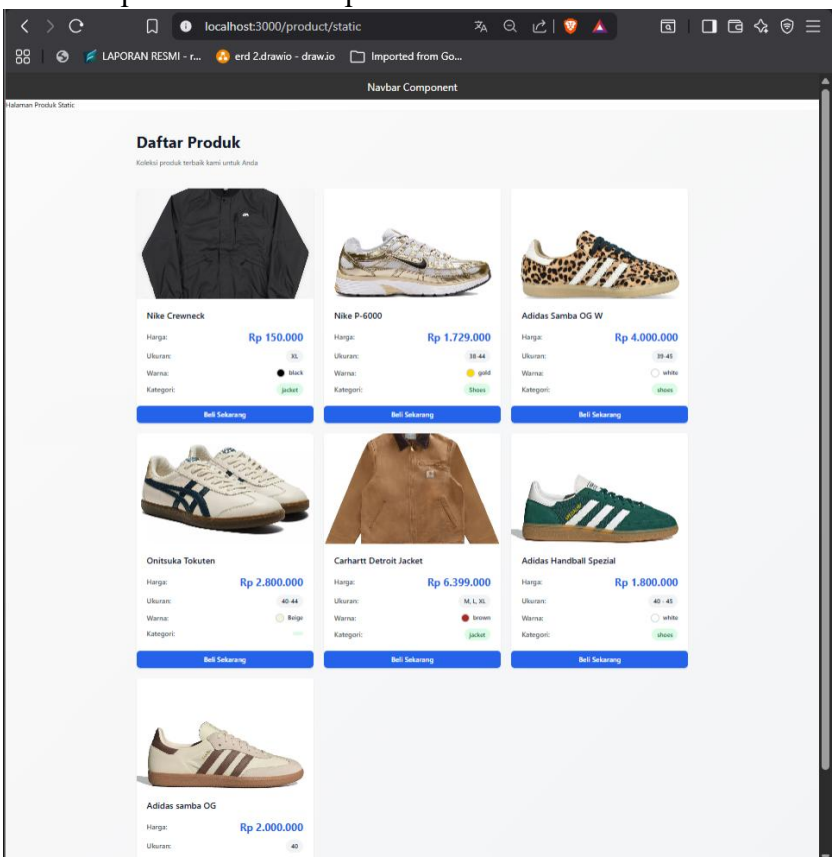
3. **SSG (*Static Site Generation*)**
   - Tambah data

     Sebelum di npm run dev dan npm run start



     Data belum terupdate jika belum di build Makanya kamu perlu menjalankan npm run dev atau npm run start (setelah build) lagi, karena proses itu akan membangun ulang (rebuild) halaman statis dan mengambil data terbaru dari database.

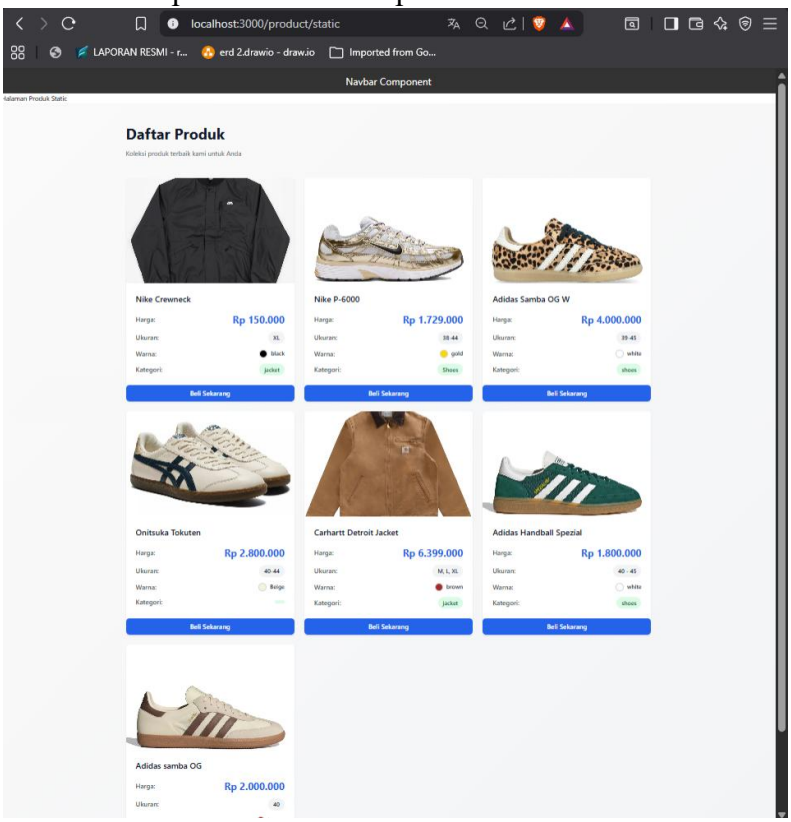
Setelah npm run dev dan npm run start



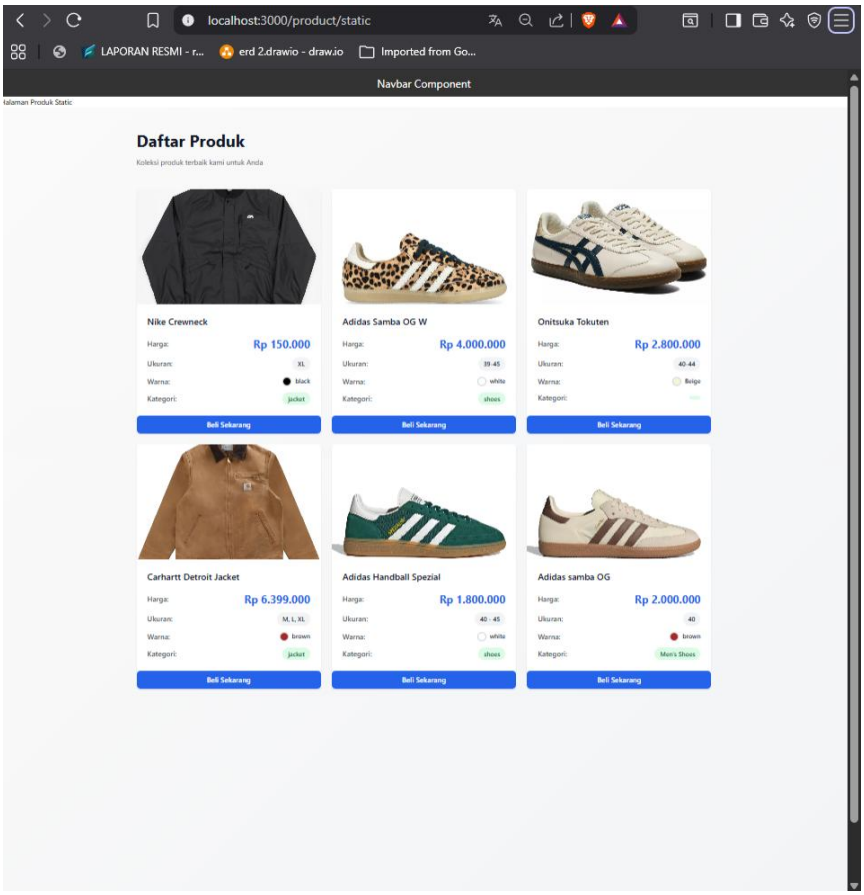
Dapat dilihat bahwa item diupdate ketike di rebuild.

- Hapus Data

Sebelum di npm run dev dan npm run start



Sesudah di rebuild dan di start



- Hasil
  SSG (Static Site Generation) di Next.js itu cara render halaman dengan mengambil data sekali saat proses build, lalu hasilnya disimpan jadi file statis (HTML). Jadi ketika kamu menambahkan data baru ke database, halaman yang sudah di-build tidak otomatis tahu kalau ada perubahan, karena dia tidak fetch ulang data setiap ada request seperti SSR.

Perbandingan Hasil dari CSR, SSR dan SSG

- **CSR (Client Side Rendering)**: Data langsung ter-update ketika halaman di-refresh karena browser melakukan request ke API setiap kali halaman dibuka. Hal ini membuat perubahan seperti tambah atau hapus data langsung terlihat, dan request API juga tampak jelas di DevTools karena proses fetch terjadi di sisi client.

- **SSR (Server Side Rendering)**: Data juga selalu ter-update saat halaman diakses atau di-refresh, tetapi proses pengambilan data dilakukan di server. Akibatnya, hasil yang diterima user sudah berupa HTML yang siap tampil, dan request API tidak terlihat di browser karena terjadi di belakang layar (server-side).

- **SSG (Static Site Generation)**: Data tidak langsung berubah meskipun ada update di database karena halaman dibuat saat proses build dan bersifat statis. Perubahan data baru akan muncul hanya setelah dilakukan build ulang (npm run build dan npm run start), sehingga SSG tidak cocok untuk data yang sering berubah tetapi unggul dalam performa karena halaman sudah di-generate sebelumnya.

STUDI ANALISIS

Jawab pertanyaan berikut:

1. **Mengapa SSG tidak menampilkan data terbaru?**
   SSG tidak menampilkan data terbaru karena data hanya diambil satu kali saat proses build, lalu disimpan menjadi file html statis. Setelah itu, halaman tidak melakukan fetch ulang ke database, sehingga perubahan data baru tidak akan terlihat sebelum dilakukan build ulang.

2. **Mengapa SSG lebih cepat?**
   SSG lebih cepat karena halaman sudah berupa file HTML statis yang siap dikirim ke browser tanpa perlu proses render ulang atau pengambilan data setiap request. Server hanya mengirimkan file yang sudah jadi, sehingga waktu loading menjadi lebih singkat.

3. **Kapan SSG tidak cocok digunakan?**
   SSG tidak cocok digunakan ketika data sering berubah atau membutuhkan update secara real-time, seperti dashboard, aplikasi monitoring, atau sistem yang datanya terus bertambah dan berubah secara dinamis.

4. **Mengapa e-commerce tidak cocok menggunakan SSG murni?**
   E-commerce tidak cocok menggunakan SSG murni karena data seperti stok, harga, dan produk sering berubah. Jika menggunakan SSG saja, perubahan tersebut tidak akan langsung terlihat oleh pengguna tanpa build ulang, sehingga bisa menyebabkan informasi yang ditampilkan tidak akurat.

5. **Apa perbedaan build mode dan development mode?**
   Build mode adalah proses menghasilkan versi production dari aplikasi yang sudah dioptimasi dan biasanya digunakan sebelum dijalankan dengan npm run start. Sedangkan development mode (npm run dev) digunakan saat pengembangan, di mana perubahan kode akan langsung terdeteksi (hot reload) tanpa perlu build ulang, tetapi performanya belum seoptimal production.